

Chapter 14. Distributed Systems Techniques and Patterns

In this chapter, we will introduce some fundamental techniques of distributed systems. Some of these are generalizations of the concurrent patterns we have already met, but others are new abstractions caused by one or more of the *fallacies of distributed computing* enumerated by [L. Peter Deutsch and James Gosling](#) in the mid-1990s.

The fallacies are:

- The network is reliable.
- Latency is zero.
- Bandwidth is infinite.
- The network is secure.
- Topology doesn't change.
- There is one administrator.
- Transport cost is zero.
- The network is homogeneous.

Some of these effects will impact us more than others—for example, the question of latency is of particular importance, as is the possibility of topology change within a cluster. In addition, architecture styles like service-oriented architecture (SOA) and microservice-based architectures can significantly increase the number of distributed connections, which, in turn, increases the potential impact of the fallacies.

That is, we may discover that although we distributed our app as a means of optimizing it and allowing it to scale, instead we did the exact opposite because we failed to take into account one or more of the fallacies. To counteract this possibility, we need some tools and techniques to help immunize us from the impact of the fallacies.

In this chapter, we will introduce several of these useful technologies and will point out which of the fallacies are relevant to each case. Our first section discusses a number of low-level, or basic, *distributed data structures*. After that, we discuss the concept of consensus, which is broadly the idea that cluster members will need to agree on some mutable data value for distributed computation to be viable.

In the second half of the chapter, we will discuss examples of libraries that can simplify the process of building distributed systems in Java. As we cover each topic, we will see how one or more of the fallacies is mitigated by each technology. Finally, we will bring the topics together and see how to enhance Fighting Animals with some of these technologies.

Basic Distributed Data Structures

To construct distributed data structures, we will start by introducing some basic building blocks. This is by no means an exhaustive list—many others are useful and relevant but these are a starting point (and many are reused in the higher-level libraries we discuss later).

Clocks, IDs, and Write-Ahead Logs

One of the simplest building blocks of distributed data structures is the [generation clock](#). This is a monotonically increasing value that is used to order events in a distributed system. It is attached to all messages sent in the system (and stored in replicated logs).

For example, it can be used in leader-follower clusters to handle the possibility of the leader being temporarily disconnected from the followers. If a new leadership election occurs, then the generation clock will advance, so the follower nodes know they should discard messages from the old leader if it reconnects to the cluster.

A technique that can be easily generalized from single-machine concurrent programming to distributed systems is the use of immutable objects. Provided that immutable objects can be serialized and deserialized, they can be sent over the network without fear. There is no possibility of lost updates—because the object state cannot be updated after it has been created.

One other basic need is for unique cluster-wide IDs. There are several techniques for generating unique IDs in a distributed system, but one of the simplest is to use a cluster-wide segment. The leader node is responsible for handing out segments to the follower nodes, which can then generate IDs from that segment and request a new segment when it is exhausted.

For example, each follower claims a segment of, for example, one million IDs to allocate. Behind the scenes, this could be done by the leader process using an `AtomicLong` and incrementing that atomic when a follower claims a segment.

After claiming the segment, the cluster member can increment a local counter each time a new ID is required. Once all IDs in the segment are used, the follower can claim a new segment.

The consequence of this approach is that network traffic is vastly reduced—almost all the ID generation can be done in memory and is extremely fast. Another consequence is that this approach scales a lot better than a raw `AtomicLong` because there is a lot less contention—it is much more efficient and faster to get segments of `1_000_000 AtomicLong` IDs from the leader than requesting them one at a time.

There will not be any duplicate IDs, but there are some issues you need to be aware of:

- IDs generated by different followers will be out of order.
- If a member goes down without fully using its segment, there might be gaps. In most cases, for ID generation, this isn't relevant.
- If the cluster leader restarts, then (in the simplest case) the ID generator is reset and will start again from 0.

The last of these issues can be mitigated by using persistent storage (e.g., a database) to record the last segment handed out. There are also alternative solutions for creating cluster-wide unique IDs, such as `java.util.UUID` or other forms of UUID, but in many cases a simple numeric ID suffices.

To conclude this section, let's introduce the *write-ahead log* (WAL). This technique is widely deployed in database systems to provide atomicity and durability (two of the *ACID properties* for databases). The log is an ordered, append-only data structure that lives on persistent storage.

It is traditionally used to implement crash recovery—when a process restarts, it can check the log and see whether the changes had been completed, or if they need to be rolled back and the log entries discarded.

For our purposes, we can think of this technique as a log of changes that a node wants to make but has only made on a speculative basis. When a completion condition is reached, the log has reached a checkpoint, applies the changes, and clears the log.

Two-Phase Commit

The two-phase commit is a very common technique in distributed programming. For example, it often appears when a transaction needs to be persisted to a database and also propagated to a messaging system. Handling this as two separate transactions would be problematic because

of the possibility that the database transaction completed, but then the messaging transaction failed.

The basic structure of the two-phase commit is to use separate *prepare* (aka *voting*) and *commit* phases. Let's walk through this in a leader-follower system.

In the prepare phase, the leader writes to a write-ahead log. The leader replicates the change over the network to the followers, which write it to their local WALs and reply with a promise—that they're ready to commit.

When sufficient followers have replied with a promise, the leader begins the commit phase. It marks the entry as final and sends out a completion message. Followers can apply their local WAL entry, and then complete their promise by sending back an acknowledgment (ACK).

Once sufficient ACKs are received, the commit is complete.

This technique has some useful advantages. First of all, it is simple and easy to understand. Secondly, it provides atomicity and consistency—the transaction either succeeds or fails, and there is no way for different nodes to end up with different results for the transaction. It also ensures liveness—eventually, we will reach a consensus, although this can take multiple iterations of the transaction.

However, two-phase commit has multiple downsides:

- It requires promises and acknowledgments from every node, and this scales linearly with the size of the cluster.
- A single error or conflict from any of the nodes causes the transaction to be aborted and retried from scratch.
- The leader waits for everybody in the group to reply, which means the overall latency is at least that of the slowest node.

The protocol also has possible blocking behavior. This is because, in the prepare phase, nodes must allocate resources needed to complete the transaction. This means that the entire cluster is waiting with an unacknowledged promise for the entry in the write-ahead log.

Until the transaction commits or rolls back, they can't accept new updates that apply to any of the affected resources.

Practical versions of two-phase commit will also need to deal with time-outs and aim to provide at least basic fault-tolerance—when one node is unavailable or slow or is temporarily unavailable (perhaps due to a GC pause). The interested reader could consult a text such as *[Designing Data-](#)*

[Intensive Applications by Martin Kleppmann](#) for a deeper treatment of this subject.

Object Serialization

Serialization in Java has a long, and somewhat inglorious, history. Broadly, it is the conversion of the state of an object instance into a byte stream. This byte stream can then be written to disk or moved across a network, where it can be *deserialized* back into an object instance.

It has been a built-in feature of the platform since Java 1.1, but the language-level mechanism has multiple severe drawbacks and is now widely recognized as being deeply flawed in the general case.

Serialization constitutes an invisible but public constructor, and an invisible but public set of accessors for your internal state.

—Brian Goetz, [“Towards Better Serialization”](#)

Built-in serialization has also contributed to a large number of security issues over Java’s long history. As a result, most application teams have opted for a library-level solution instead of relying on the built-in mechanism.

For example, JSON is one attractive choice for a serialization format, for several reasons:

- It is extremely simple.
- It is human-readable.
- Libraries exist for it in basically every programming language.

However, this is counter-balanced by some other factors—such as the size of the serialized objects, which can contribute to poor performance for larger messages. For high-performance applications, binary encodings are preferable—and there are a great many to choose from, including [Protocol Buffers](#) and [Avro](#).

The tradeoff and architectural choices for data serialization relate to two of the fallacies we met at the start of the chapter: “Transport cost is zero” and “Bandwidth is infinite.” Performance of serialization libraries has been an important aspect of the overall design of distributed systems for many years.

For example, more than twenty years ago, one of the authors was doing performance work on service architectures where the primary gating factors for performance were:

- The serialization and deserialization of Java objects to XML
- Finding the correct granularity of services (largely to reduce the amount of the above)

While the on-the-wire serialization format you work with is probably not XML anymore, the stated general principles remain as major concerns for modern systems.

Data Partitioning and Replication

We met partitioning briefly in [Chapter 10](#), as a problem that can be detected by observability. In this section, we'll introduce it properly and place it in context as a key technique for distributed systems.

In its simplest form, data partitioning (aka *sharding*) is the dividing of a large dataset into smaller, more manageable subsets—the *partitions*. Each partition contains a subset of the overall data, and they are distributed across multiple nodes.

There are a couple of different ways that the data can be partitioned. For example, *horizontal partitioning*, dividing the data based on rows, and *vertical partitioning*, dividing the data based on columns.

In the case of horizontal partitioning, there are several well-known techniques for splitting the data into partitions. For example, the data can be partitioned by some key—an ID or a hash of important fields. As we will discuss later, this is the approach taken by systems such as Apache Kafka.

One very common approach is to use a combination of heartbeating network connections between hosts in the cluster, along with a *quorum* system for ensuring that a distributed transaction has sufficient “votes” to complete (more about this in the next section).

For fault tolerance, there may be multiple copies of each partition to mitigate the effects of a node becoming unavailable (this is the “The network is reliable” fallacy). The most obvious default is one synchronous copy, but other strategies are possible—such as two synchronous copies to permit a rolling update across the cluster while still protecting against unexpected failure of a node.

For example, we could choose to deploy multiple synchronous backups, but this would come at the cost of higher memory consumption across the cluster (as each backup consumes memory equivalent to the size of the original data structure).

However, it is important to realize that all synchronous backups require locking between the copies to prevent a distributed version of the *Lost*

Update antipattern we met in [Chapter 13](#). The more backup copies are held, the more costly this locking becomes.

In some applications, persisting an in-memory data store to a backing store can be advantageous. However, this durability comes with a potential performance problem—as for full reliability, each change to the in-memory data has to be “written through” to the physical store before the change can be fully confirmed. This is similar to the case of main memory versus cache, which we discussed in the previous chapter.

There are plenty of use cases where this behavior is overkill, and a short window of potential data loss can be tolerated by the application architecture. Under these circumstances, many systems can be configured to “write-behind,” in which case the writes to backing store effectively become asynchronous.

Observability can also be beneficial when deploying replication and partitioning. In addition to ensuring that data continues to flow through the pipeline, observing repartitioning events—not only that they occur, but also their duration—can be early indicators that the application is nearing the capacity constraints of the messaging infrastructure.

Repartitioning is *at best* linear in the number of nodes (and is sometimes as bad as quadratic). Being able to observe repartitioning across the cluster is important, because the perspective of any given node may be insufficient to form a complete overall picture.

CAP Theorem

The [CAP theorem](#) is one of the most recognizable pieces of theory in distributed systems. The name comes from the three initials for behavioral aspects of a system—consistency, availability, and partition-tolerance.

The theorem indicates that only two of the three aspects are possible for any system. Different technologies may make different choices, for example, an AP system chooses availability and partition-tolerance.

The theorem relates to two of the fallacies of distributed computing: “The network is reliable” and “Topology doesn’t change.” In fact, a network partition is a classic example of a topology change, and hence, a lack of reliability.

One, somewhat loose, way to approach the theorem is to consider that if you have a network that may drop messages, then you cannot have both complete availability and perfect consistency in the event of a partition.

So, in the event of a network partition where nodes remain up and connected to different groups of clients, one strategy is to give up consistency and remain available while partitioned. This is the same concept that we met as the split-brain in [“Repartitioning and “Split-Brain””](#).

The effect for the user in the event of a partition would be that clients connected to one partition would see locally consistent results. However, clients connected to different partitions would not necessarily see the same result. For example, a supposedly atomic value could now potentially have different values in different partitions.

Another approach is to note that network partitions—indicated by total loss of messages—are very rare on modern LANs (or within data centers). As a result, many people consider the CAP theorem to apply only over wide-area (and possibly global) networks.

If the system’s clients are always made aware of a list of cluster nodes they could connect to, in the event of loss of a datacenter (or region), clients would simply reconnect to unaffected nodes.

Consensus Protocols

In this section, we will briefly discuss two of the most commonly used *consensus protocols*. These protocols are used in distributed systems to ensure that coordinating cluster members agree on some mutable data value (the consensus) that is needed during computation.

NOTE

The use of consensus algorithms relate to the same two fallacies as we saw for the CAP theorem, i.e., “The network is reliable” and “Topology doesn’t change.”

Our treatment will build upon the discussion of the two-phase commit earlier in the chapter. There is also an [excellent presentation from Devnexus 2023](#) that covers both algorithms in simple language.

This area also provides a case study of another component of successful observability: knowing what any cluster transition events look like.

Based on this, you should configure observability not to alert until there is an actual problem. This may not be at the first occurrence—these transition events are relatively rare, but they do occur naturally in normal operation. For example, when monitoring systems like ZooKeeper, there should never be two consecutive monitoring intervals (minutes, by default) during which multiple nodes report being a leader.

Paxos

Paxos is a popular voting-based consensus algorithm (or, more properly, a family of algorithms) used in many systems, such as Cassandra, DynamoDB, and Chubby (a distributed lock system from Google).¹ It was introduced by Leslie Lamport in the 1980s and uses some concepts that we have already met (such as the generation clock).

The protocol has two basic roles that cluster nodes can perform:

- Proposer
- Acceptor

In general, messages are identified with a *round number* N (which is the generation clock time), and they may also have a to-be-agreed-upon value.

Paxos has two phases, each with two subphases:

Phase 1a: Prepare

Proposer creates a `Prepare` message and sends it to all known Acceptors.

Phase 1b: Promise

When any Acceptor receives a `Prepare` message from any Proposer, it returns a `Promise` to that Proposer—essentially, that it agrees to honor a value to be provided by that Proposer.

Phase 2a: Accept

Once a Proposer has received promises from a quorum of Acceptors, it needs to set a value v to its proposal. It does so by sending `Accept` messages, denoted by the pair (N, v) , to all Acceptors that have promised to honor the proposal.

Phase 2b: Accepted

If an Acceptor receives an `Accept` from a Proposer, it must accept it if and only if it has not already promised (in Phase 1b) to only consider proposals having a higher round number (i.e., a number greater than N).

Paxos is a quorum-based protocol, so a simple majority is enough to move forward—this is to ensure that forward progress can still be made even if some nodes are disconnected, slow, or overwhelmed. One critical part of the protocol is the acceptance criteria—this is designed to prevent conflicting values from disrupting convergence to a consensus.

Paxos has been very well studied and has many different possible extensions. One of the most obvious is to include a third role—the *Learner* (sometimes known by other names, such as *observer* in ZooKeeper).

Learners are passive observers of the cluster—they don't propose new values, and they don't vote in the ballot. Instead, they're just notified about the outcome when the value is agreed upon—and both Acceptors and Proposers can notify Learners once the value is fixed. This can be very useful for reducing read load on the nodes participating in the consensus algorithm.

NOTE

In practice, many Paxos implementations merge the roles, so an Acceptor can also be a Learner.

We should also mention one of the most significant theoretical results about consensus algorithms at this point—known as the Fischer, Lynch, and Paterson (FLP) Impossibility result.

Their 1985 [paper](#) (which has a very readable [simplified overview](#)) establishes that in an asynchronous system, consensus algorithms will provide only two of the three properties of safety, liveness, and fault tolerance.

This is a strong result, but a number of workarounds have been developed to minimize its restrictions. For example, introducing a short random delay between proposed values would reduce the chances of a liveness failure (essentially a distributed LiveLock similar to the one we met in [Chapter 13](#)), but the underlying theoretical possibility cannot be completely ruled out.

Let's move on to look at the other consensus algorithm—Raft.

Raft

When [Raft](#) was created, one of its major design goals was to provide a [simple and easy-to-understand](#) consensus algorithm. It approaches this by decomposing the consensus problem into two distinct subproblems:

- Leader election
- Log replication

To further promote the idea of simplicity, Raft is based on the idea that every node has only two possible election states (Stable and Election), and three possible role states (Leader, Follower, and Candidate). It's a leader-

follower based protocol, so only one of the nodes will be a Leader, and every other node is a Follower, at least while an election is not running.

Every update goes through the Leader, which writes to its log and asynchronously replicates the change to its Followers. Once the Leader has successfully replicated the data to a quorum, the commit index is advanced, the operation is applied to the state machine, and the update is deemed successful.

Followers in Raft are totally passive; they just accept the updates sent to them by the Leader. Leader nodes in Raft send out heartbeat messages to all Followers at regular intervals (e.g., 100 ms). If a Follower notices that the Leader is no longer sending heartbeats, then it can send out a message to its peers and call for an election, as we will see.

Like Paxos, Raft is based on the idea of generation clocks, which are also referred to as *terms*. Every node in the cluster knows which node is the Leader and what the current term number is. It also has a list of the Followers (which are peers of one another).

The two messages are `AppendEntry` to send an update from Leader to Followers (it also serves as the heartbeat message, by just sending a null update payload), and `RequestVote`, which is sent to all peers if a Follower has not received an `AppendEntry` within a timeout.

The `AppendEntry` message, and its response, can be represented in Java like this:

```
record AppendEntry (
    int term,
    int leaderId,
    List<Payload> payloads,
    int prevLogIndex,
    int prevLogTerm,
    int leaderCommit) { }

record AppendEntryResponse (
    int term,
    boolean accepted,
    int conflictIndex,
    int conflictTerm) { }
```

with the `RequestVote` and `RequestVoteResponse` looking like this:

```
record RequestVote (
    int term,
    int candidateId,
    int lastLogIndex,
```

```

    int lastLogTerm
  ) {}

  record RequestVoteResponse(
    int term,
    boolean inFavor
  ) {}

```

The resulting state machine for Raft can be seen in [Figure 14-1](#).

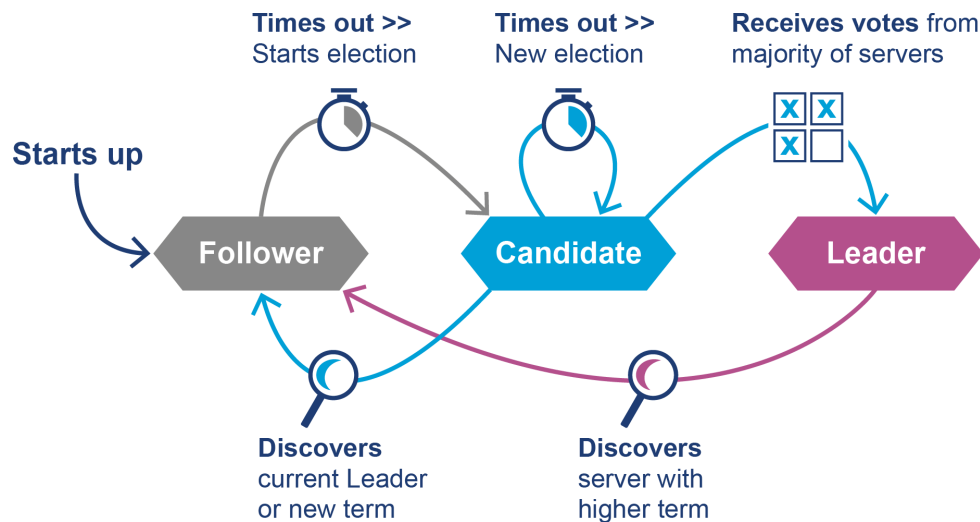


Figure 14-1. Raft state machine

Note that, when the cluster starts, there’s no Leader node—so every node starts as a Follower. To handle this, nodes can self-promote to Candidate—to do so, they increment the term number and send every other node a `RequestVote`.

Raft has been extensively studied, and the algorithm has been shown to [not fully guarantee liveness](#) in the event of a network fault without additional modifications. This means that it’s still possible an election can theoretically continue forever.

However, Raft minimizes this probability by introducing a random election timeout (and that a Candidate cannot win an election unless its log is up to date). In practice, this means that LiveLock does not occur, but just as for Paxos, we cannot prove it is impossible—another consequence of the FLP result.

Finally, we should note that just because an algorithm is correct does not mean the implementation code is also correct. For Raft, one of the possible Java implementations is the version provided by the [jgroups-raft project](#)—which is used in products like Red Hat’s Infinispan—and has been [extensively tested for correctness bugs](#).

Distributed Systems Examples

In this section, we will discuss some case studies of how these distributed techniques are used in practice by real systems. Our examples are basically infrastructural, in that they are designed to enable the programmer to separate the distributed systems’ aspects from their business logic. Let’s get started by discussing an example of a distributed database.

Distributed Database—Cassandra

[Cassandra](#) is one of the most popular open source NoSQL distributed databases. It is written in Java and is nonrelational and column-oriented. By default, Cassandra is an AP (available, partition-tolerant) database, but this consistency can be configured on a per-query level.

It is relatively common for NoSQL systems to display this type of *eventual consistency* behavior. Cassandra provides the concept of a *consistency level*, which specifies how many replicas nodes must acknowledge the write before the coordinator reports success to the client. Some common values of the consistency level are:

- `ONE` : Only a single acknowledgement from any replica is sufficient to confirm the write.
- `LOCAL_QUORUM` : Acknowledgement from a simple majority of nodes in the same datacenter is needed to confirm.
- `QUORUM` : Acknowledgement from a simple majority of cluster nodes globally is needed to confirm.
- `ALL` : Acknowledgement from all nodes is required.

These options are ordered from fastest to slowest. Cassandra is optimized for writing large amounts of data quickly, so the more stringent the consistency level, the slower the writes will become.

Note that typical deployments of Cassandra may be globally distributed, so `QUORUM` and `ALL` may well involve WAN traffic, which can have write-confirmation times in the tens or hundreds of milliseconds (due to speed-of-light limitations).

TIP

The `ALL` setting is not recommended, as it is not only the slowest but also affects availability—a single failed node can cause writes to fail. This goes against the design of Cassandra as an AP database, and if you are tempted to use it, then this can be an “architecture smell” of something wrong somewhere else in the system design.

The primary interface into the Cassandra DB is Cassandra Query Language (CQL). This is a query language that is somewhat similar to SQL (despite Cassandra being a NoSQL database), in that both have the same basic concept of a table constructed of columns and rows. However, Cassandra is nonrelational, so it does not support joins or subqueries.

If we look at some basic examples of CQL, then it can be deceptively similar to SQL. For example, all of these examples are *both* valid SQL and valid CQL:

```
CREATE TABLE IF NOT EXISTS demoTable (id INT PRIMARY KEY);
ALTER TABLE demoTable ADD newField INT;
CREATE INDEX myIndex ON demoTable (newField);
INSERT INTO demoTable (id, newField) VALUES (1, 2);
SELECT * FROM demoTable WHERE newField = 2;
SELECT COUNT(*) FROM demoTable;
DELETE FROM demoTable WHERE newField = 2;
```

However, just as this statement:

```
var x = 15;
```

is both valid Java and JS but has substantial semantic difference in the two environments, it is important not to take too much from surface impressions and simple examples.

There is actually a substantial amount of difference between CQL and SQL, despite the identical syntax. For example, `INSERT` and `DELETE` both behave very differently in Cassandra than is typical in SQL databases.

It is also true that what we’re referring to as a table is more properly called a *columnfamily* in CQL—the alias “table” is really there to make new arrivals with SQL experience more comfortable. Likewise, a Cassandra *keyspace* is a namespace that defines how data is replicated on nodes and is broadly analogous to a database schema for a SQL RDBMS.

Keyspaces are designed to control data replication for a set of tables. This replication is controlled on a per-keyspace basis, so data that has different replication requirements typically resides in different keyspace. As a consequence of this, a cluster will typically have one keyspace per application that the cluster serves.

Note that there are cases where Cassandra’s default of eventual consistency just doesn’t work—i.e., sometimes you need to maintain a strict order of reads and writes.

For these cases, [*linearizable consistency*](#) is useful, and Cassandra has two sorts of CQL operations—regular (which we’ve already met) and *lightweight transactions*, which provide linearizable consistency for conditional updates.

The lightweight transaction syntax is different from that of regular operations, and it uses an implementation of Paxos to support this feature. In their simplest form, lightweight transactions are CAS operations that look something like `INSERT ... IF NOT EXISTS`.

NOTE

Normal operations should not be mixed with lightweight transactions—if your application is an appropriate use case for lightweight transactions, then they should be used throughout.

The SERIAL consistency level is used for lightweight transactions and ensures that conditional writes are isolated and applied atomically. Cassandra lightweight transactions are, therefore, only really lightweight when compared to transaction strategies in other DB types or to two-phase commit.

In-Memory Data Grid—Infinispan

[*Infinispan*](#) is a distributed cache and key-value NoSQL in-memory data store developed by Red Hat as the successor to JBoss Cache. As well as being in-memory, Infinispan can also persist data to a more permanent *cache store*. These are pluggable, and many different implementations exist to allow Infinispan data to be persisted in an existing system.

The technology can be used to add clustering and high availability to libraries and frameworks. By delegating state management to Infinispan, the internal distributed data structures that Infinispan provides can provide clustering capability.

The `jgroups` library is a useful toolkit that provides reliable, group network communication for Infinispan and many other projects. It provides many useful features, such as node discovery, both point-to-point and point-to-multipoint, failure detection, and data transfer.

The fault tolerance capability is based on the `jgroups-raft` component, which (as the name suggests) is an implementation of the Raft protocol. Infinispan provides a configurable replication factor, working as an AP system, similar to Cassandra.

Infinispan exposes a `Cache` interface, which extends `java.util.Map`—so, from an end-user perspective it can be seen, in its simplest terms, as a “big `HashMap` that can extend across machines”—Infinispan also supports non-JVM platforms as clients.

Transactional use cases—both JTA and XA—are also supported, and Infinispan can participate in distributed transactions brokered by a JTA transaction manager. Transactions in Infinispan are optional and can be disabled for higher performance, depending on your application.

For observability, Infinispan has good support for tracing, including OpenTelemetry traces—despite not being based on HTTP, the jgroups protocol has been extended to include OTel support.

Event Streaming—Kafka

[Apache Kafka](#) is a key piece of infrastructure technology that provides a stream of events in a publish-subscribe paradigm. The events are used both as triggers for actions and the means of distributing state to nodes. This is a common pattern for implementing microservice architectures.

Unlike other messaging systems (such as JMS and AMQP), Kafka was built with scale and throughput as the primary design goals. It is a client-broker system, so messages are sent from clients to clusters of Kafka broker processes.

In addition to authorization, Kafka provides the capability to authenticate at transport level, and this mitigates the fallacy “The network is secure.”² Of course, to fully mitigate this issue, we would also need to include authentication on all of our microservices as well.

The heart of Kafka is an ordered, replicated log. This allows efficient implementation of both reads and writes, as shown in [Figure 14-2](#).

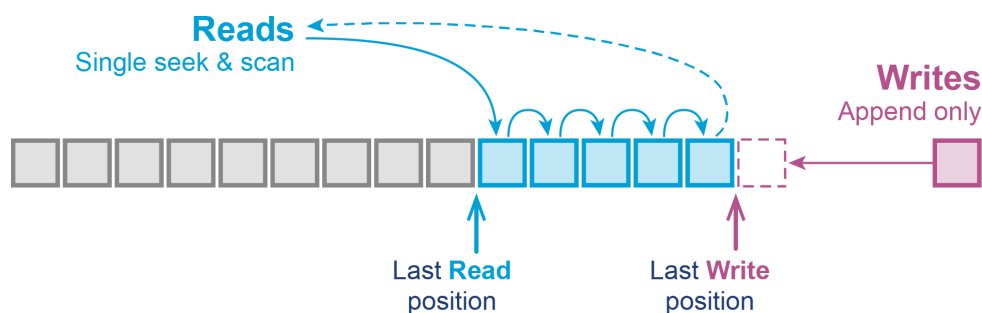


Figure 14-2. Kafka reads and writes

The bouncing arrow represents a seek-and-scan read, and note that there are no updates—Kafka is append-only. This means that reads and writes are sequential operations. In turn, this allows them to work with the linear nature of underlying media (e.g., memory and disk) to leverage things

like prefetch and memory caches as well as providing an opportunity to batch operations together.

Kafka processors subscribe to *topics*, which are always divided into one or more partitions. Topics are a useful abstraction that is essentially a label and common configuration for a set of partitions. The partitions have two properties:

- Each partition is consumed by exactly one consumer.
- A single consumer can consume more than one partition.

Therefore, the number of active consumers must be less than or equal to the number of partitions.³ These properties enable Kafka to provide both ordering guarantees and load balancing over a pool of consumers.

NOTE

Kafka guarantees the order of events within the same partition. However, it does not (by default) guarantee a total order of events across all partitions.

The key class to examine is `ProducerRecord` in the package `org.apache.kafka.clients.producer`, which has these fields:

```
public class ProducerRecord<K, V> {
    private final String topic;
    private final Integer partition;
    private final Headers headers;
    private final K key;
    private final V value;
    private final Long timestamp;

    // ...
}
```

In most deployments the destination partition will be computed from a hash of the key. This allows Kafka to guarantee that messages having the same key always land in the same partition, and therefore, are always in order.

However, it is possible to change this behavior—either by explicitly providing the partition number or by just allowing Kafka to round-robin the messages. Note that in this final case, different messages with the same key may well be handled by different consumers, which can destroy the ordering properties.

Kafka used to use a standalone ZooKeeper service but has switched to its own implementation of Raft—known as KRaft. This somewhat simplifies adopting Kafka, as it reduces the number of components that need to be deployed, maintained, and observed.

Let’s move on to discuss a concrete example—and think about how we might use some of these technologies to enhance Fighting Animals.

Enhancing Fighting Animals

First of all, let’s introduce a `VeterinaryHospital` service for any animals that might have been injured in any of the battles. The hospital’s purpose is to take care of any animal that has been fighting in too many battles.

We could use any of the technologies from the last section to implement the hospital service. For example:

- Existing services could write to a shared instance of Cassandra when one of their animals fights.
- Infinispan could store all currently available animals in a collection of maps, and each existing service could use that instead of holding a local map of animals.
- Existing services could communicate with the hospital via Kafka.

We could also consider ideas like using `jgroups-raft` to implement a cluster of our existing web services, but as they are effectively stateless, this is almost certainly overkill—instead, we should rely on K8s to provide our load balancing and related concerns.

Let’s start with the third option, and implement an initial version of `VeterinaryHospital` that receives updates via Kafka. We can iterate on this initial design later in the chapter.

Introducing Kafka to Fighting Animals

We need three pieces to build out to introduce Kafka. Obviously, we will need some Kafka infrastructure in the form of a cluster of Kafka brokers. We also need the hospital service—note that it will still be a microservice, but it will differ from all of our existing services, because it does not communicate over HTTP but instead listens for messages on Kafka topics.

In addition, we have to modify each of the existing “leaf” microservices (those that return an animal, so `Feline`, `Fish`, and `Mustelid`) so that they will publish a Kafka message that says which animal was returned. Each

microservice will have its own topic, so the hospital will need to listen to them all.

We can see the new layout of components in [Figure 14-3](#).

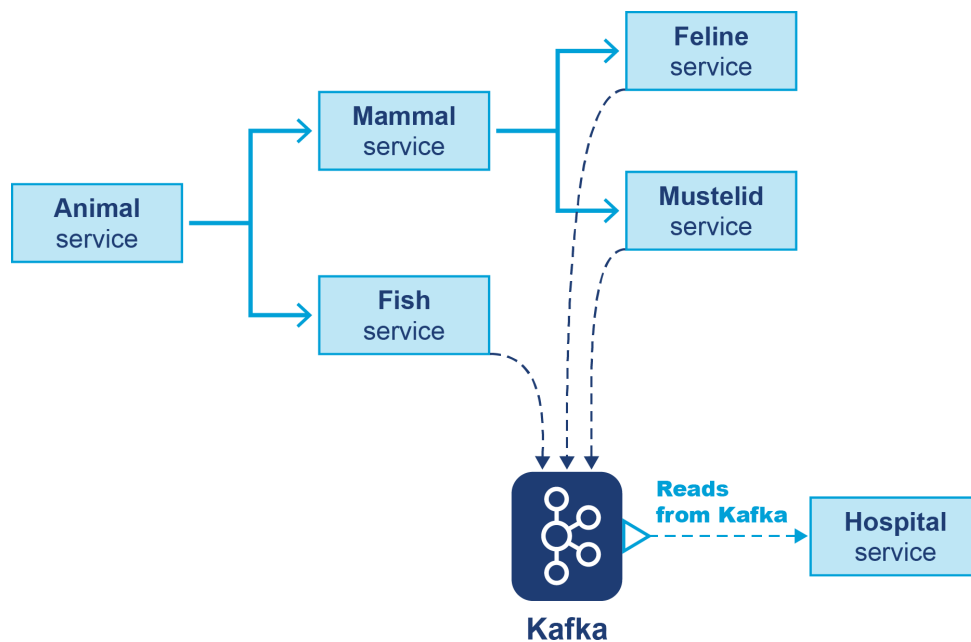


Figure 14-3. Fighting Animals with Kafka

To implement this, first of all, we'll need some Kafka infrastructure. The `distributed_systems` branch of the Fighting Animals repo has the code and config changes. We're using the [Kafka Docker image](#) from the [Debezium project](#), and once again, we're using a pinned version rather than floating (like `latest`).

We have the new Kafka nodes set up like this in `docker-compose.yml` (and similarly for `kafka-2` and `kafka-3`):

```
# Kafka
kafka-1:
  image: debezium/kafka:2.7.0.Final
  ports:
    - "19092:9092"
    - "19093:9093"
  environment:
    - CLUSTER_ID=g4xWbaRgd-b-zQYgISlrY5
    - NODE_ID=1
    - KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka-1:9093,2@kafka-2:9093,3@kafka-
```

and we also need to make the leaf node services depend upon Kafka, like this:

```
# Feline service
feline-service:
  image: feline_demo:latest
  ports:
```

```

- "8085:8085"
depends_on:
- kafka-1
- kafka-2
- kafka-3

```

We're also using the newest (at the time of writing, August 2024) version, which supports the [KRaft protocol](#) rather than requiring a separate ZooKeeper cluster. KRaft is significantly more performant than external ZooKeeper and is intended to become the default for Kafka in time.

There is now a lot more noise in the startup log using Docker Compose, and startup (and shutdown) now takes significantly longer. We could have avoided some of this noise by using a single-node Kafka cluster, but we feel it's more informative to be able to see some of the details of clustering, leadership elections, and KRaft as the application starts up and runs.

Next, let's look at the code changes. Each of the leaf services needs to notify Kafka of each choice by producing and sending a message to a suitable topic. The Feline service has been modified like this (and the changes to Fish and Mustelid are similar):

```

@RestController
public class FelineController {
    private final List<String> CATS = List.of("tabby", "jaguar", "leopard");

    private final KafkaProducer<String, String> producer;

    public FelineController() {
        Properties properties = new Properties();
        properties.put("bootstrap.servers", "kafka-1:9092"); // PLAINTEXT
        properties.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
            StringSerializer.class);
        properties.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
            StringSerializer.class);
        producer = new KafkaProducer<>(properties);
    }

    @GetMapping("/getAnimal")
    public String makeBattle() throws InterruptedException {
        // Random pause
        Thread.sleep((int) (20 * Math.random()));

        // Return random cat (and also send to Kafka)
        var cat = CATS.get((int) (CATS.size() * Math.random()));
        var key = UUID.randomUUID().toString();
        var producerRecord = new ProducerRecord<>("FELINE", key, cat);
        producer.send(producerRecord);
        return cat;
    }
}

```

```
}  
}
```

The changes to the constructor are just to set the Kafka producer to allow messages to be sent to the broker. We use the Kafka `ProducerRecord` class, as it allows us to specify the topic as well as the key and value for the message.

When we send the first requests to the Kafka-enabled animal service, we see lines like this in the log:

```
kafka-1-1          | 2024-07-14 09:13:19,275 - INFO [data-plane-kafka-request-  
handler-7:Logging@66] - Sent auto-creation request for Set(MUSTELID) to the  
active controller.  
kafka-1-1          | 2024-07-14 09:13:19,275 - INFO [quorum-controller-1-event-  
handler:ReplicationControlManager@670] - [QuorumController id=1] CreateTopic:  
result(s): CreatableTopic(name='MUSTELID', numPartitions=1, replicationFactor=1, assignments=[], configs=[]): SUCCESS  
kafka-1-1          | 2024-07-14 09:13:19,276 - INFO [quorum-controller-1-event-  
handler:ReplicationControlManager@457] - [QuorumController id=1] Replicate  
TopicRecord for topic MUSTELID with topic ID cnzLVYp8Q007FIUd6mUkzg.
```

This shows the Kafka topics and partitions being auto-created and set up, so our modified services are connecting and sending, although we don't log the outgoing messages. Next, we need the Hospital service, so something is listening and responding to the message flow.

A Simple Hospital Service

The hospital has been implemented as a simple Kafka-based Quarkus service. This slightly complicates the setup, because it is usually easier to have Spring Boot and Quarkus components in separate Maven projects.⁴

Don't worry if you're not that familiar with Quarkus—one of the points of this example is to show how simple it can be. We won't use anything that requires any specific Quarkus knowledge—just Maven and Docker.

A skeleton for the service can be generated by installing the Quarkus CLI tool and then using the `create app` command, [specifying the Kafka extension](#). The project derived from this, which we'll use for these examples, is [available on GitHub as the “Fighting Animals Hospital”](#). For now, we'll be using the `listen_and_log` branch.

Our hospital is just one class:

```

@ApplicationScoped
public class VeterinaryHospital {

    public static final String FISH_CHANNEL = "fish";
    public static final String FELINE_CHANNEL = "feline";
    public static final String MUSTELID_CHANNEL = "mustelid";

    @PostConstruct
    public void init() {}

    void onStart(@Observes StartupEvent ev) {
        Log.infof("Hospital starting up");
    }

    @Incoming(FISH_CHANNEL)
    @Incoming(FELINE_CHANNEL)
    @Incoming(MUSTELID_CHANNEL)
    public CompletionStage<Void> processMainFlow(Message<String> message) {
        var payload = message.getPayload();
        var topic = message.getMetadata(IncomingKafkaRecordMetadata.class)
            .get().getTopic();

        Log.infof("Processed: %s on topic %s, benching them for the next round
            payload, topic);

        return message.ack();
    }
}

```

Overall, the code should be easy to understand. We have a single `@ApplicationScoped` class, which is going to act as a controller and respond to Kafka messages. We have included a couple of callback hooks—`init()` and `onStart()`—which don't really do anything in this example except point out two useful points in the service lifecycle where a more complex service might want to take some action.

The code in our message handler `processMainFlow()` is deliberately a little more complex than a simple “Hello World” Kafka service would be. In particular, Quarkus offers several different ways to interact with Kafka, and in our example, we're using manual control of the Kafka ACK, via the final `message.ack()`.

This is more complex than the simplest case would be but also less “magical.” The return type (`CompletionStage<Void>`) of our message handler method also gives us a clue that this is intended to fit into part of a larger flow.

WARNING

If a service uses manual Kafka ACKs, then you must ensure that every possible path through the method (including any exceptional paths) performs a `message.ack()` before exiting the method. Failing to do this can result in the Kafka consumer position not advancing, and so the consumer may end up re-consuming the message (including after a restart).

Also note the repeated `@Incoming` annotation—this arises because we want the same code to handle multiple incoming topics. We use the `getMetadata()` method on the message, so we can extract the topic the message was received on—a service that listened to multiple topics but didn't care which topic a message was received on wouldn't need to do this, of course.

We don't usually look at the `import` statements of our examples, but in this case, it's interesting to note how a few Quarkus-specific classes are being used:

```
import io.quarkus.logging.Log;
import io.quarkus.runtime.StartupEvent;
import io.smallrye.reactive.messaging.kafka.api.IncomingKafkaRecordMetadata;
import jakarta.annotation.PostConstruct;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.event.Observes;
import org.eclipse.microprofile.reactive.messaging.Incoming;
import org.eclipse.microprofile.reactive.messaging.Message;

import java.util.concurrent.CompletionStage;
```

One aspect that should be briefly discussed is that this service does *not* use the standard Kafka consumer classes. Instead, it uses the Kafka support provided by Smallrye and Microprofile *reactive messaging*.

However, this is largely an implementation detail. The Quarkus framework deals with the reactive and nonblocking aspects and leaves the developer free to focus on just implementing a handler that processes `Message` objects as they arrive.

The hospital service also needs some config, which is in `src/main/resources/application.properties`:

```
# Kafka bootstrap applies to all topics
kafka.bootstrap.servers=kafka-1:9092,kafka-2:9092,kafka-3:9092

# Kafka boilerplate
kafka.sasl.jaas.config = ""
```

```

kafka.sasl.mechanism = PLAIN
kafka.security.protocol = PLAINTEXT

# Input queue
mp.messaging.incoming.fish.connector=smallrye-kafka
mp.messaging.incoming.fish.topic=FISH
mp.messaging.incoming.feline.connector=smallrye-kafka
mp.messaging.incoming.feline.topic=FELINE
mp.messaging.incoming.mustelid.connector=smallrye-kafka
mp.messaging.incoming.mustelid.topic=MUSTELID

# Logging
quarkus.log.level=INFO

```

There is a small amount of additional complexity here, because we want the hospital to listen to multiple topics. To achieve this, we specify each one as a separate `mp.messaging.incoming` stanza in our config, and then use a corresponding repeated annotation in the code.

As the hospital service needs to be brought up as part of our cluster, we have to include it in the main `docker-compose.yml` file in the `fighting-animals` repo, like this:

```

# Hospital
hospital-service:
  image: hospital_demo:latest
  ports:
    - "8000:8000"
  depends_on:
    - kafka-1
    - kafka-2
    - kafka-3

```

This makes the deployment a little more complex, as we must build the jars and containers from both the main project and the hospital project before running `docker compose up`.

An Active Hospital

The next step is to make the hospital service actually do something. A hospital that can't do anything while the casualties continue to arrive is of extremely limited use.

Our next iteration is to add the capability for the hospital to tell the other services that an animal is injured and should not be included in the next fight.

We're going to do this by using Infinispan as a remote cache. The code is on the `with_infinispan` branch and can be seen in [Figure 14-4](#).

We're also going to dial down the Kafka cluster to a single node—just to make it easier to see what's going on. We saw the extra complexity and startup latency that was induced by the multinode Kafka cluster in the last section, and now we want to focus on the changes the new Infinispan component brings in.

NOTE

If you have previously used a multinode Kafka cluster, then when you switch to the branch with a single node, you will need to purge the multinode config. You can do this by switching to a non-Kafka branch (e.g., `main`) and running `docker compose up --remove-orphans` to completely remove the old Kafka containers and then switching to the single-node branch.

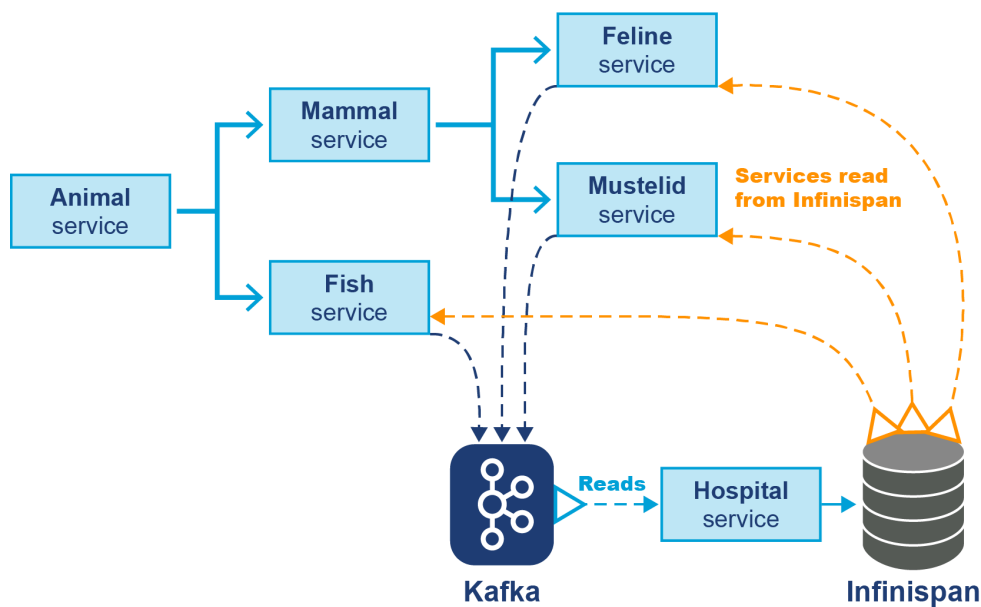


Figure 14-4. The full Fighting Animals system

To get started, we'll add this dependency to the hospital service:

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-infinispan-client</artifactId>
</dependency>
```

and we also need a code change to the `VeterinaryHospital` class:

```
@Inject
@Remote("animals")
RemoteCache<String, String> lastBattledAnimal;
```

This will store the last animal from each clade that was involved in a battle. We only need to include the single line:

```
lastBattledAnimal.put(topic, payload);
```

in the `processMainFlow()` method before calling `message.ack()`. We also need some cache configuration, which is stored partly in `application.properties`:

```
# Infinispan
quarkus.infinispan-client.cache.images.configuration-uri=animals.yaml
quarkus.infinispan-client.devservices.port=11222
quarkus.infinispan-client.hosts=infinispan-1:11222
quarkus.infinispan-client.username=xxxxxx
quarkus.infinispan-client.password=yyyyyy
```

and in the `animals.yaml` file (the name is chosen to coincide with the annotation and the application property):

```
replicatedCache:
  mode: "SYNC"
  statistics: "true"
  encoding:
    key:
      mediaType: "application/x-protostream"
    value:
      mediaType: "application/x-protostream"
  locking:
    isolation: "REPEATABLE_READ"
  indexing:
    enabled: "true"
    storage: "local-heap"
    startupMode: "NONE"
```

We need to make changes to the main Fighting Animals project as well. To start with, we need to modify the controller classes for all the leaf services, like this example from `FelineController`:

```
@GetMapping("/getAnimal")
public String makeBattle() throws InterruptedException {
    // Random pause
    Thread.sleep((int) (20 * Math.random()));

    // Look up last injured animal
    var injured = cacheManager.getCache(InfinispanConfiguration.CACHE_NAME)
        .get("FELINE");

    String cat;
```

```

do {
    Thread.sleep(1);
    cat = CATS.get((int) (CATS.size() * Math.random()));
    System.out.printf("Looking up uninjured cat - is %s OK?", cat);
} while (injured == null || injured.equals(cat));

// Return random uninjured cat (and also send to Kafka)
var key = UUID.randomUUID().toString();
var producerRecord = new ProducerRecord<>("FELINE", key, cat);
producer.send(producerRecord);
return cat;
}

```

The code uses a simple `do` loop to keep selecting a random cat until we choose one that is uninjured (i.e., is not in the cache maintained by the hospital service). This relies on a `RemoteCacheManager` that we inject into the controller classes:

```

@Autowired private RemoteCacheManager cacheManager;

```

We also need a configuration class to set up the animals cache on this side:

```

@Configuration
public class InfinispanConfiguration {
    public static final String CACHE_NAME = "animals";

    @Bean
    @Order(Ordered.HIGHEST_PRECEDENCE)
    public InfinispanRemoteCacheCustomizer caches() {
        return b -> {
            URI cacheConfigUri;
            try {
                cacheConfigUri = this.getClass().getClassLoader()
                    .getResource("animals.xml").toURI();
            } catch (URISyntaxException e) {
                throw new RuntimeException(e);
            }

            b.remoteCache(CACHE_NAME).configurationURI(cacheConfigUri);
        };
    }
}

```

We also need some configuration in `application.properties`, like this:

```

infinispan.remote.server-list=infinispan-1:11222
infinispan.remote.auth-username=xxxxxxx
infinispan.remote.auth-password=yyyyyy
infinispan.remote.marshaller=org.infinispan.commons.marshall.ProtoStream ↵
Marshaller

```

and a cache config (in our case, `animals.xml`):

```

<?xml version="1.0"?>
<distributed-cache name="animals" mode="SYNC" statistics="false">
  <encoding media-type="application/x-java-object"/>
  <indexing enabled="false" />
</distributed-cache>

```

With all these pieces in place, the full system should now start:

```

infinispan-1-1 | 2024-07-22 13:47:19,215 INFO (main) [org.infinispan. ↵
CONTAINER] ISPN000104: Using EmbeddedTransactionManager
infinispan-1-1 | 2024-07-22 13:47:19,445 INFO (ForkJoinPool.commonPool- ↵
worker-2) [org.infinispan.server.core.telemetry.TelemetryServiceFactory] ↵
ISPN000953: OpenTelemetry integration is disabled
infinispan-1-1 | 2024-07-22 13:47:19,475 INFO (main) [org.infinispan.SER ↵
ISPN080018: Started connector Resp (internal)
infinispan-1-1 | 2024-07-22 13:47:19,491 INFO (ForkJoinPool.commonPool- ↵
worker-1) [org.infinispan.SERVER] ISPN080018: Started connector HotRod (int
infinispan-1-1 | 2024-07-22 13:47:19,544 INFO (ForkJoinPool.commonPool- ↵
worker-2) [org.infinispan.SERVER] ISPN080018: Started connector REST (inte
infinispan-1-1 | 2024-07-22 13:47:19,559 INFO (main) [org.infinispan.SER ↵
Using transport: Epoll
infinispan-1-1 | 2024-07-22 13:47:19,640 INFO (main) [org.infinispan.SER ↵
ISPN080004: Connector SinglePort (default) listening on 0.0.0.0:11222
infinispan-1-1 | 2024-07-22 13:47:19,641 INFO (main) [org.infinispan.SER ↵
ISPN080034: Server '8239b401a660-45518' listening on http://0.0.0.0:11222
infinispan-1-1 | 2024-07-22 13:47:19,676 INFO (main) [org.infinispan.SER ↵
ISPN080001: Infinispan Server 14.0.21.Final started in 10421ms

```

The architecture we have built is partially asynchronous—HTTP requests hit the microservices, and then data is handed off via Kafka to the hospital service, which updates the in-memory cache. This design forces us to consider the possibility of *partial availability*—what happens if the hospital is unavailable? Should the animal services continue, or does the failure of the hospital mandate the unavailability of others?

In this context, because the possibility exists of requests coming in so quickly that a hospitalized animal will still be selected, then arguably, the failure of the hospital should not cause a total outage. From this perspec-

tive, the same should also be true of the infrastructure components (i.e., Kafka and Infinispan).

Before we move on, it is important to once again highlight the role of observability—it is absolutely crucial that these distributed infrastructure components be observable. We need to be able to make sense of what’s happening in all the components of our cloud app, including the infrastructure components.

For clarity, we disabled OpenTelemetry support in the Infinispan node, but in a real production system, this would be essential. The ability to observe, and also correlate, behaviors between our services and components is not only very powerful but also increasingly important.

Summary

In this chapter, we introduced some basic building blocks for building distributed systems—both distributed generalizations of single-machine concurrent data structures and genuinely new structures.

We outlined the two most common consensus protocols—Paxos and Raft. We have looked at some case studies of common cloud infrastructure components (all built in Java) that use the consensus algorithms and other low-level concepts to provide highly available solutions to common application and architectural problems. To bring it all together, we showed how to use two of these technologies to enhance our Fighting Animals example and discussed the tradeoffs that naturally arise when doing so.

In the next, and final, chapter we will look ahead and show you what new developments in the Java and JVM world will have the most impact on cloud-deployed applications in the coming years.

- 1** In addition, ZooKeeper uses the ZAB protocol, which can be thought of as a Paxos variant that is specifically tailored to ZooKeeper’s use case.
- 2** This capability enables some useful fine-grained patterns—such as granting consumers read-only access only to a specific topic (or topics), and granting the producer only write access, with no consumer allowed to see the contents of any topic not explicitly granted to it. This is very helpful in multitenant environments and for GDPR type work.
- 3** It is possible to have additional consumers that are idle and are present to provide hot standbys.
- 4** It is entirely possible to have the two technologies coexist in the same Maven build, but it requires separate submodules, and Fighting Animals hasn’t been set

up that way, so the extra complexity is not worth it for our example.